



Docker

Docker Compose

Erfan Taheri

Imperative vs Declarative

► Imperative Configuration

- You **specify how** to reach the desired state
- Step-by-step instructions (commands, scripts)
- Example:

```
$ docker run -d --name web -p 8080:80 nginx  
$ docker network create mynet  
$ docker network connect mynet web
```

Imperative vs Declarative

▶ Declarative Configuration

- ▶ You **specify what** the final state should look like
- ▶ Tool figures out the steps automatically
- ▶ Example (docker-compose.yml):

```
services:  
  web:  
    image: nginx  
    ports:  
      - "8080:80"  
    networks:  
      - mynet
```

YAML Refresher - What is YAML?

► Definition

- **YAML = YAML Ain't Markup Language**
- **A human-readable data serialization format**
- **Commonly used for configuration files (Docker Compose, Kubernetes, Ansible, CI/CD pipelines)**

► Characteristics

- **Uses indentation (spaces) instead of braces/brackets**
- **Supports scalars (strings, numbers, booleans), lists, and mappings (key: value)**
- **Comments start with #**
- **Designed to be easy for humans to read & write**

YAML Refresher - data types

Scalars (basic values - key value pair)

```
name: "nginx"  
replicas: 3  
enabled: true  
timeout: null
```

Mappings / Dictionary

```
service:  
  image: nginx  
  ports: "8080:80"
```

Lists (Sequences / Arrays)

```
ports:  
  - "8080:80"  
  - "443:443"  
addresses: ["1.2.3.4", "5.6.7.8"]
```

Nested Structures

```
services:  
  web:  
    image: nginx  
    volumes:  
      - ./app:/usr/share/nginx/html
```

YAML Refresher - Basics, Anchors & Aliases

```
# YAML basics: mapping, sequence, scalar (indentation matters)
app:
  name: myapp
  replicas: 3

# Anchors and aliases to avoid repetition
db-config: &dbcfg
  user: app
  password: s3cr3t

services:
  db:
    image: postgres:15
    environment: *dbcfg
  replica:
    image: postgres:15
    environment: *dbcfg

# Merge key example (<<)
common: &common
  restart: unless-stopped
web:
  <<: *common
  image: nginx
```

History of Docker Compose

2013 Fig → 2014 Docker Compose v1 → 2017 v2 → 2021 CLI Plugin

▶ Origins

- ▶ **2013–2014:** Docker Inc. acquired **Fig** (a Python-based tool for defining and running multi-container apps).
- ▶ Fig became the foundation of **Docker Compose**.

▶ Early Docker Compose

- ▶ Initially written in **Python**.
- ▶ Provided a **YAML-based declarative config** for multi-service apps.
- ▶ Simplified orchestration for local development.

▶ Evolution

- ▶ **v1 (2015):** Simple CLI for docker-compose up/down.
- ▶ **v2 (2017–2020):** Rewritten for better performance, support for Docker Swarm.
- ▶ **2021+ (v2.x):** Migrated into **Docker CLI plugin** (docker compose instead of docker-compose).
- ▶ Introduced **Compose Specification** → unified format adopted by Docker, Kubernetes, Podman, ECS, ACI.

Docker Compose Sample

```
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
    networks:
      - test

  db:
    image: mysql:8.0
    command: --default-authentication-plugin=mysql_native_password
    environment:
      MYSQL_DATABASE: mydb
      MYSQL_USER: user
      MYSQL_PASSWORD: password
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - test

volumes:
  db_data:

networks:
  test:
```

Service Definition

- ▶ **image / build (context, dockerfile, args, target)**
- ▶ **ports (host:container, short vs long syntax) and mode=host/published for swarm**
- ▶ **volumes: type=bind, volume, tmpfs; propagation and read-only flags**
- ▶ **environment vs env_file vs args; variable substitution and .env precedence**
- ▶ **healthcheck (test, interval, timeout, retries, start_period) - use for depends_on health**
- ▶ **restart policies (no, on-failure, unless-stopped, always)**
- ▶ **resource limits (deploy.resources for swarm; mem_limit/cpu_shares sometimes supported)**

Build Options & BuildKit Integration

```
services:
  myapp:
    build:
      context: ./app
      dockerfile: Dockerfile
      args:
        NODE_ENV: production
        API_URL: "https://api.example.com"
    image: myuser/myapp:latest
    ports:
      - "8080:8080"
    environment:
      - ENV=production
    networks:
      - appnet
```

Networks in Compose - Overview

- ▶ **Compose creates a default network per project (project name prefix) if none defined**
- ▶ **Define custom networks with driver, ipam config, and attach services to multiple networks**
- ▶ **Compose networks are simple for single-host setups; for multi-host use overlay or a CNI**
- ▶ **DNS-based service discovery: services can reach each other by service name**

Networks in Compose - Options

Type	Definition Example	Use Case
driver	Defines the network driver (bridge, host, overlay, macvlan, etc.)	driver: bridge
driver_opts	Custom options to pass to the driver (key-value map).	driver_opts: { com.docker.network.bridge. name: custom0 }
attachable	Allow standalone containers to connect to the network.	attachable: true
internal	Restricts external access (network isolated from outside).	internal: true
labels	Metadata for networks (useful for organization & automation).	labels: ["env=prod", "team=backend"]
external	Use an existing Docker network instead of creating a new one.	external: true
name	Custom name for the network (instead of auto-generated).	name: custom_net
ipam	Configure IPAM (IP Address Management) for custom subnets, gateways, ranges.	ipam: { config: [{ subnet: "172.16.238.0/24" }] }

Volumes in Compose - Types & Options

Type	Definition Example	Use Case
named volume	<code>volumes:_[SEP] dbdata:_[SEP] driver: local</code>	Databases, stateful services
bind mount	<code>volumes:_[SEP] - ./data:/var/lib/data</code>	Dev, host integration, device access
tmpfs	<code>volumes:_[SEP] - type: tmpfs,tmpfs:_[SEP] size: 100m</code>	Ephemeral in-memory storage
external volume	<code>volumes:_[SEP] logs:_[SEP] external: true</code>	Shared volumes managed outside compose

Configs & Secrets - Compose Usage

Feature	Compose Support	Notes
configs	v3.3+ (works with swarm and some compose implementations)	Non-secret config files, injected as files
secrets	v3.x for swarm; docker compose supports secrets with limitations	Use secret managers for production; avoid env vars for secrets
.env files	Supported by compose CLI	Used for variable substitution; do not store secrets in .env for prod

Profiles, Overrides & Multiple Compose Files

- ▶ Use multiple -f files: `docker compose -f compose.yml -f compose.override.yml up -d` (later files override earlier)
- ▶ `compose.override.yml` is automatically applied if present
- ▶ Profiles let you enable subsets of services: profiles: ['worker'] and `'docker compose --profile worker up'`
- ▶ Use x-extensions and anchors to avoid repetition and create reusable building blocks
- ▶ Avoid secrets in override files; prefer external/sealed secret mechanisms in CI

Best Practices - Docker Compose

Area	Recommendation	Why
Minimize build context	Add <code>.dockerignore</code> , COPY only necessary files	Faster builds, smaller images
Prefer images for prod	Build in CI, push to registry, use image: in compose	Repeatable deployments, smaller surface on hosts
Healthchecks + depends_on	Use healthcheck and wait-for scripts; avoid naive depends_on	Ensure services are ready before starts
Use named volumes	Persist DB data in volumes not in container writable layer	Durability & easier backups
Secrets & configs	Use secret managers or Compose secrets; avoid env var secrets	Reduce leak risk
Resource limits	Set <code>deploy.resources</code> when using orchestration; test on target infra	Prevent noisy neighbors
Immutable tags	Avoid latest in production; use semver/tags/digests	Predictability & rollback safety

Compose - Useful Commands & Troubleshooting

Command	Purpose
<code>docker compose up -d --build</code>	Start services, rebuild images
<code>docker compose ps</code>	List services & state
<code>docker compose logs -f web</code>	Follow logs
<code>docker compose exec web sh</code>	Run commands inside container
<code>docker compose down --volumes</code>	Tear down and remove created volumes
<code>docker compose config</code>	Validate merged config and see final interpolated YAML

CI/CD - Example: Build & Push with Buildx (GitHub Actions)

```
name: CI Build
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: docker/setup-buildx-action@v2
      - name: Login
        uses: docker/login-action@v2
        with:
          registry: ghcr.io
          username: ${ secrets.GITHUB_ACTOR }
          password: ${ secrets.GITHUB_TOKEN }
      - name: Build and push
        uses: docker/build-push-action@v4
        with:
          push: true
          tags: user/repo:latest
          cache-from: type=registry,ref=user/repo:cache
          cache-to: type=registry,ref=user/repo:cache,mode=max
```

Summary & Next Steps

- ▶ **YAML anchors reduce repetition and improve maintainability**
- ▶ **Use BuildKit and CI for reproducible image builds; prefer pushing images for production**
- ▶ **Define clear network & volume strategies; use secrets & configs properly**
- ▶ **Use compose config and healthchecks to validate and orchestrate startup order**
- ▶ **Practice labs: multi-file overrides, profiles, Compose in CI with buildx cache**